Exercise 3: Zero-Copy Clone + Dynamic Tables

Objective

Use **Zero-Copy Cloning** to create an instant copy of your database, then use **Dynamic Tables** inside the clone to build a pipeline that refreshes automatically when source data changes.

Prerequisites

- Exercise 2 completed (`AIRBNB.DEV` with `raw_listings` [8 rows], `raw_hosts` [4 rows], `raw_reviews` [10 rows])

About Zero-Copy Cloning

Cloning creates an instant copy of a database, schema or table **without duplicating the underlying data**. The clone shares the same micro-partitions as the original.

When you modify data in the clone, only the affected micro-partitions are copied (copy-on-write). Key consequences:

- Cloning a 1 TB database takes **seconds**, not hours
- The clone uses **zero additional storage** until you modify data
- Changes to the clone **do not affect** the original

Step 1: Set your context

```
USE ROLE ACCOUNTADMIN;
USE WAREHOUSE WH_DBT_DEV;
USE DATABASE AIRBNB;
USE SCHEMA DEV;
```

Step 2: Clone the entire database

```
CREATE DATABASE AIRBNB_STAGING CLONE AIRBNB;
```

Verify both exist:

```
SHOW DATABASES LIKE 'AIRBNB%';
```

Expected: both `AIRBNB` and `AIRBNB_STAGING` .

Check that the clone has the same data:

```
SELECT COUNT(*) AS listings_in_clone FROM AIRBNB_STAGING.DEV.raw_listings;
```

Expected: **8 rows** — identical to the original.

Step 3: Prove the clone is independent

Delete a row from the **clone**:

```
DELETE FROM AIRBNB_STAGING.DEV.raw_listings WHERE id = 1008;

SELECT COUNT(*) AS clone_count    FROM AIRBNB_STAGING.DEV.raw_listings;  -- 7
SELECT COUNT(*) AS original_count FROM AIRBNB.DEV.raw_listings;          -- 8
```

The clone has 7 rows but the original is untouched (still 8). That is copy-on-write in action.

Undo the delete so the clone matches the original again:

```
INSERT INTO AIRBNB_STAGING.DEV.raw_listings
  SELECT * FROM AIRBNB.DEV.raw_listings WHERE id = 1008;

SELECT COUNT(*) FROM AIRBNB_STAGING.DEV.raw_listings;  -- 8
```

About Dynamic Tables

In S10 you used dbt to transform data: you write a SQL model and run `dbt run` manually or on a schedule.

Dynamic Tables are Snowflake's declarative alternative: you define the desired result as a SQL query and Snowflake keeps it up to date automatically.

The key concept is `TARGET_LAG` — how stale the data is allowed to be. You do not say *"refresh every hour"*. You say *"this data should never be more than 1 hour old"*. Snowflake decides when to refresh.

> *We will create Dynamic Tables **in the clone** (`AIRBNB_STAGING`) so they do not interfere with the dbt exercise (Ex4) that uses `AIRBNB.DEV` .*

Step 4: Switch context to the clone

```
USE DATABASE AIRBNB_STAGING;
USE SCHEMA DEV;
```

Verify:

```
SELECT CURRENT_DATABASE(), CURRENT_SCHEMA();  -- AIRBNB_STAGING, DEV
```

Step 5: Create a staging Dynamic Table

This is the equivalent of `src_listings.sql` from S10, but as a Dynamic Table:

```
CREATE OR REPLACE DYNAMIC TABLE src_listings_dt
  TARGET_LAG = '1 minute'
  WAREHOUSE = WH_DBT_DEV
```

```
AS
  SELECT
      id                      AS listing_id,
      name                    AS listing_name,
      host_id,
      neighbourhood,
      room_type,
      minimum_nights,
      price::DECIMAL(10,2)    AS price_usd,
      created_at,
      updated_at
  FROM raw_listings;
```

What each part means:

- `TARGET_LAG = '1 minute'` : data must never be more than 1 minute stale
- `WAREHOUSE = WH_DBT_DEV` : warehouse used for refresh queries
- `AS SELECT ...` : the query that defines the table contents

Step 6: Create a second staging Dynamic Table

```
CREATE OR REPLACE DYNAMIC TABLE src_hosts_dt
  TARGET_LAG = '1 minute'
  WAREHOUSE = WH_DBT_DEV
AS
  SELECT
      id      AS host_id,
      name    AS host_name,
      CASE WHEN is_superhost = 't' THEN TRUE ELSE FALSE END AS is_superhost,
      created_at,
      updated_at
  FROM raw_hosts;
```

Step 7: Create a mart Dynamic Table that joins both

```
CREATE OR REPLACE DYNAMIC TABLE fct_listings_dt
  TARGET_LAG = '1 minute'
  WAREHOUSE = WH_DBT_DEV
AS
  SELECT
      l.listing_id,
      l.listing_name,
      l.neighbourhood,
      l.room_type,
      l.minimum_nights,
      l.price_usd,
      h.host_name,
      h.is_superhost
```

```
  FROM src_listings_dt l
  LEFT JOIN src_hosts_dt h ON l.host_id = h.host_id;
```

> *`fct_listings_dt` references `src_listings_dt` and `src_hosts_dt`. Snowflake auto-detects this dependency and refreshes them in the right order — like dbt's `ref()`, but without you writing anything explicit.*

Step 8: Query the Dynamic Tables

Wait ~30-60 seconds for the initial refresh, then run:

```
SELECT * FROM fct_listings_dt ORDER BY price_usd DESC;
```

Expected: **8 rows** with listing + host info joined — same result as the dbt `fct_listings` model from S10.

Step 9: Insert new data and watch the auto-refresh

The key test: add a row to the **raw** table and watch the mart update:

```
INSERT INTO raw_listings VALUES
  (1009, 'New Apartment in Born', 101, 'Born', 'Entire home/apt', 2, 150, '2024-06-
01', '2025-04-10');
```

Verify the raw table has 9 rows:

```
SELECT COUNT(*) FROM raw_listings;  -- 9
```

Wait 60-90 seconds (for the 1-minute target lag).

Check the Dynamic Table:

```
SELECT COUNT(*) FROM fct_listings_dt;  -- 9
```

The mart updated automatically without you running any command:

```
SELECT listing_name, price_usd, host_name
FROM fct_listings_dt
WHERE listing_id = 1009;
```

Expected:

LISTING_NAME	PRICE_USD	HOST_NAME
New Apartment in Born	150.00	Maria Garcia

The entire pipeline — from raw data to joined mart — refreshed automatically.

Step 10: Check the refresh history

```
SELECT
    name,
    state,
    refresh_action,
    refresh_start_time,
    refresh_end_time,
    DATEDIFF('second', refresh_start_time, refresh_end_time) AS duration_seconds
FROM TABLE(INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY())
ORDER BY refresh_start_time DESC
LIMIT 10;
```

You can see each refresh cycle: when it ran, how long it took, whether it was INCREMENTAL or FULL.

Step 11: Manual refresh, suspend and resume

Force an immediate refresh (skip the target lag wait):

```
ALTER DYNAMIC TABLE fct_listings_dt REFRESH;
```

Pause automatic refresh (e.g., during maintenance):

```
ALTER DYNAMIC TABLE fct_listings_dt SUSPEND;
```

Resume:

```
ALTER DYNAMIC TABLE fct_listings_dt RESUME;
```

Step 12: Clean up the clone

We are done with `AIRBNB_STAGING`. Since it was a clone, dropping it frees zero storage (the shared micro-partitions remain with the original):

```
USE DATABASE AIRBNB;
USE SCHEMA DEV;

DROP DATABASE AIRBNB_STAGING;

SHOW DATABASES LIKE 'AIRBNB%';  -- only AIRBNB remains
```

Checklist

- [ ] Cloned `AIRBNB` to `AIRBNB_STAGING` instantly
- [ ] Verified clone independence (delete in clone did not affect original)

- [ ] Created 3 Dynamic Tables with 1-minute lag
- [ ] Saw the mart Dynamic Table auto-refresh when upstream data changed
- [ ] Cleaned up the clone

Key Concepts

Concept	What it means
Zero-Copy Clone	Instant copy of a DB/schema/table without duplicating data
Copy-on-write	Cloned data is only physically copied when modified
Dynamic Table	Auto-refreshed table defined by a query + `TARGET_LAG`
TARGET_LAG	Max allowed staleness; Snowflake decides when to refresh
Refresh history	`INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY`
Auto dependency	Snowflake builds the refresh DAG by parsing the SQL

Questions

1. You clone a 5 TB database. How much additional storage does the clone use immediately after creation?
2. If `src_listings_dt` has `TARGET_LAG = '1 minute'` and `fct_listings_dt` also has `TARGET_LAG = '1 minute'`, what is the maximum end-to-end latency from raw data change to `fct_listings_dt` being updated?
3. A Dynamic Table uses `WH_DBT_DEV` for refresh. What happens to the automatic refresh if the warehouse is suspended?
4. When would you pick Dynamic Tables over dbt? When the opposite?